(Supplementary Information)
Multi-Task Optimization over Networks of Tasks

Julian Hatzky[1], Thomas Bartz-Beielstein[2], A. E. Eiben[1], and Anil Yaman[1]

[1] Vrije Universiteit Amsterdam, Amsterdam, The Netherlands
{j.hatzky, a.e.eiben, a.yaman}@vu.nl
[2] TH Köln, Cologne, Germany
thomas.bartz-beielstein@th-koeln.de

1 Node Coverage Guarantees Under Random Sampling

If we draw at each evaluation step a random node and perform an action, we may ask how many evaluations are required so that we visit each of the ν nodes at least once with high certainty.

This problem can be framed as a *coupon collector's problem* [6]. Writing M for the number of draws, the expected number of steps needed to visit all ν nodes at least once is

$$\mathbb{E}[M] = \nu H_\nu = \nu \left(\ln \nu + \gamma + O\left(\frac{1}{\nu}\right)\right),$$

where H_ν is the ν-th harmonic number and $\gamma \approx 0.57721$ is the Euler-Mascheroni constant.

Beyond the expectation, one may also ask for a high-probability guarantee. Using the asymptotic limit for the coupon collector's completion time, the probability that all nodes have been visited by draw

$$M = \nu(\ln \nu + c)$$

converges to

$$\mathbb{P}\big(M \leq \nu(\ln \nu + c)\big) \approx e^{-e^{-c}}.$$

To achieve at least 99% probability of having visited all nodes, we solve for c

$$\begin{aligned} e^{-e^{-c}} &= p \\ \ln\left(e^{-e^{-c}}\right) &= \ln(p) \\ -e^{-c} &= \ln(p) \end{aligned}$$

$$\text{For } p = 0.99: \quad c = -\ln\big(-\ln(0.99)\big) \approx 4.60015 \approx \ln(100).$$

Since each node is selected independently with probability $\frac{1}{\nu}$ at every draw, the visit count of any particular node over M draws follows a Binomial distribution,

$$X_i \sim \text{Binomial}\big(M, \tfrac{1}{\nu}\big)\,,$$

from which the expected number of visits per node is

$$\mathbb{E}[X_i] = \frac{M}{\nu}.$$

We evaluate four environments with differing numbers of distinct tasks and a fixed amount of draws (number of evaluations) $M = 1{,}000{,}000$:

Table 1: Coupon collector bounds for the four benchmark environments with $M = 1{,}000{,}000$ total evaluations.

Environment	ν	$\mathbb{E}[M]$	$M_{0.99}$	$\mathbb{E}[X_i]$
Archery / Arm / Cartpole	5 000	45 472	65 612	200
Hexapod	2 000	16 356	24 412	500

2 MONET Algorithm

Algorithm 1 and Algorithm 2 detail the two learning subroutines invoked in Algorithm **??**.

Individual Learning. When individual learning is selected, the current elite at node $\boldsymbol{\tau}_i$ is mutated independently of any neighbor. If no solution exists at the node, a random solution is drawn from $\mathcal{U}(p_{\min}, p_{\max})$ and evaluated. Otherwise, Gaussian mutation with standard deviation σ_m scaled by the parameter range is applied. The offspring replaces the current elite only if its fitness is at least as high.

Algorithm 1 Individual Learning

Require: Task $\boldsymbol{\tau}$, archive $\mathcal{A}$, bounds $p_{\min}, p_{\max}$, mutation std σ_m
Ensure: Updated $\mathcal{A}[\boldsymbol{\tau}]$

```
if A[τ] = ∅ then                                   ▷ initialize empty node
    θ ~ U(p_min, p_max)                            ▷ random solution
    A[τ] ← (θ, f(θ, τ))
    return
end if
θ ← A[τ].solution                                  ▷ current elite
ε ~ N(0, σ_m(p_max − p_min))                       ▷ Gaussian perturbation
y ← clip(θ + ε, p_min, p_max)                      ▷ mutant offspring
f_y ← f(y, τ)
if f_y ≥ A[τ].fitness then                         ▷ elitist replacement
    A[τ] ← (y, f_y)
end if
```

Social Learning. When social learning is selected, the current elite is recombined with a solution drawn from a neighboring node. If the current node is empty, the neighbor's solution is copied as an initial seed (or a random solution is used when no neighbor has a solution). Recombination is performed via Simulated Binary Crossover (SBX) [5] with distribution index $\eta_c = 10$ [4]. As with individual learning, the offspring replaces the elite only upon improvement.

Algorithm 2 Social Learning

Require: Task $\boldsymbol{\tau}$, neighbor task $\boldsymbol{\tau}_j$, archive $\mathcal{A}$, bounds $p_{\min}, p_{\max}$
Ensure: Updated $\mathcal{A}[\boldsymbol{\tau}]$

```
if A[τ] = ∅ then                                   ▷ initialize empty node
    if A[τ_j] ≠ ∅ then
        θ ← A[τ_j].solution                        ▷ copy from neighbor
    else
        θ ~ U(p_min, p_max)                        ▷ random fallback
    end if
    A[τ] ← (θ, f(θ, τ))
    return
end if
if A[τ_j] = ∅ then
    return                                         ▷ no valid neighbor solution
end if
y ← SBX(A[τ].solution, A[τ_j].solution, η_c = 10)  ▷ crossover
y ← clip(y, p_min, p_max)
f_y ← f(y, τ)
if f_y ≥ A[τ].fitness then                         ▷ elitist replacement
    A[τ] ← (y, f_y)
end if
```

3 Closed-Form Task Fitness Definitions

This appendix gives the explicit fitness expressions for the four benchmarks.

3.1 Archery

With initial velocity vector

$$\mathbf{v} = v_0 \begin{bmatrix} -\sin(\text{yaw}) \\ \cos(\text{yaw})\cos(\text{pitch}) \\ \cos(\text{yaw})\sin(\text{pitch}) \end{bmatrix}, \quad v_0 = 70\,\text{m/s},$$

the target-plane hit time is $t_{\text{hit}} = D/v_y$ whenever $v_y > 0$; if $v_y \leq 0$ the shot misses and $f = 0$. Taking $g = 9.8\,\text{m/s}^2$ and wind $W \in [-10, 10]\,\text{m/s}^2$ along x, the miss coordinates on the target plane ($y = D$) are

$$m_x = \tfrac{1}{2}\,W\,t_{\text{hit}}^2 + v_x\,t_{\text{hit}}, \qquad m_z = -\tfrac{1}{2}\,g\,t_{\text{hit}}^2 + v_z\,t_{\text{hit}}, \qquad r = \sqrt{m_x^2 + m_z^2}.$$

Fitness is computed on a 10-ring target of ring width 0.061 m:

$$f = \frac{\max\bigl(0,\ 10 - \lfloor r/0.061 \rfloor\bigr)}{10} \in \{0, 0.1, \ldots, 1.0\}.$$

3.2 Arm

With end-effector position $\mathbf{p}_{\text{ee}} \in \mathbb{R}^2$ and a fixed target $\mathbf{p}_{\text{tar}} = (0.5, 0.5)$,

$$f = \exp(-\|\mathbf{p}_{\text{ee}} - \mathbf{p}_{\text{tar}}\|_2) \in (0, 1].$$

The joint commands are $\mathbf{q} = (\boldsymbol{\theta} - \frac{1}{2}) \cdot 2\pi\,\alpha_{\max}/10$ where $\boldsymbol{\theta} \in [0, 1]^{10}$ is the solution vector and $\alpha_{\max}$ is the task-dependent maximum joint-angle fraction; each of the 10 links has length $L/10$ with L the task-dependent total arm length. $\mathbf{p}_{\text{ee}}$ is the end-effector position returned by forward kinematics.

3.3 Cartpole

The task uses the `CartPole-v1` dynamics from OpenAI Gym with the pole mass m_p and pole length L replaced by the per-task values; the default cart mass m_c is retained. With pole angle ϕ, cart position x_c, control force $F \in \{-10, +10\}$ N, and gravity $g = 9.8\,\text{m/s}^2$,

$$\ddot{\phi} = \frac{g\sin\phi + \cos\phi\left(-F - m_p L\dot{\phi}^2\sin\phi\right)/(m_c + m_p)}{L\left(\frac{4}{3} - \frac{m_p\cos^2\phi}{m_c + m_p}\right)}, \qquad \ddot{x}_c = \frac{F + m_p L\left(\dot{\phi}^2\sin\phi - \ddot{\phi}\cos\phi\right)}{m_c + m_p}.$$

A solution is evaluated over $n_{\text{rollouts}} = 10$ rollouts with distinct Gym seeds. Each rollout runs until the pole leaves the balanced region ($|\phi| \geq 12^\circ$ or $|x_c| \geq 2.4\,\text{m}$) or until the step cap $t_{\max} = 1000$ is reached. Writing $t_{\text{balanced}}^{(k)}$ for the number of surviving steps on rollout k, the reported fitness is the arithmetic mean

$$f = \frac{1}{n_{\text{rollouts}}} \sum_{k=1}^{n_{\text{rollouts}}} \min\bigl(t_{\max},\ t_{\text{balanced}}^{(k)}\bigr) \in [0, t_{\max}].$$

3.4 Hexapod

With gait parameters $\boldsymbol{\theta} \in [0,1]^{36}$ and morphology $\boldsymbol{\tau} \in \mathcal{T}$ instantiated through a per-task URDF, a PyBullet simulation is run for up to 3 s of wall-clock simulator time. The episode terminates early if any body-axis angle (roll, pitch, yaw) exceeds $\pi/8$ or if the lateral deviation satisfies $|y| > 0.5\,\mathrm{m}$. Writing $p_x(t)$ for the forward position of the center of mass and $T \leq 3\,\mathrm{s}$ for the time of termination or episode end,

$$f(\boldsymbol{\theta}, \boldsymbol{\tau}) = p_x(T),$$

reported in metres. The controller returns $f = 0$ if any entry of $\boldsymbol{\theta}$ lies outside $[0,1]$.

4 Detailed Sensitivity Analysis

This appendix provides per-domain SHAP beeswarm plots and the SPOT-tuned hyperparameter configurations that complement the global sensitivity analysis.

4.1 Per-Domain SHAP Analysis

Figures 1 and 2 show the SHAP beeswarm plots for the archery and arm domains, respectively. Each point represents one MONET run; its horizontal position indicates the SHAP value (contribution to the predicted metric), and its color encodes the feature value (red = high, blue = low).

In both domains, p_{ind} exhibits the strongest effect: high values of p_{ind} (more individual learning) consistently decrease predicted fitness, corroborating the global finding that social learning is the primary driver of performance. The remaining parameters show smaller, more task-specific effects.

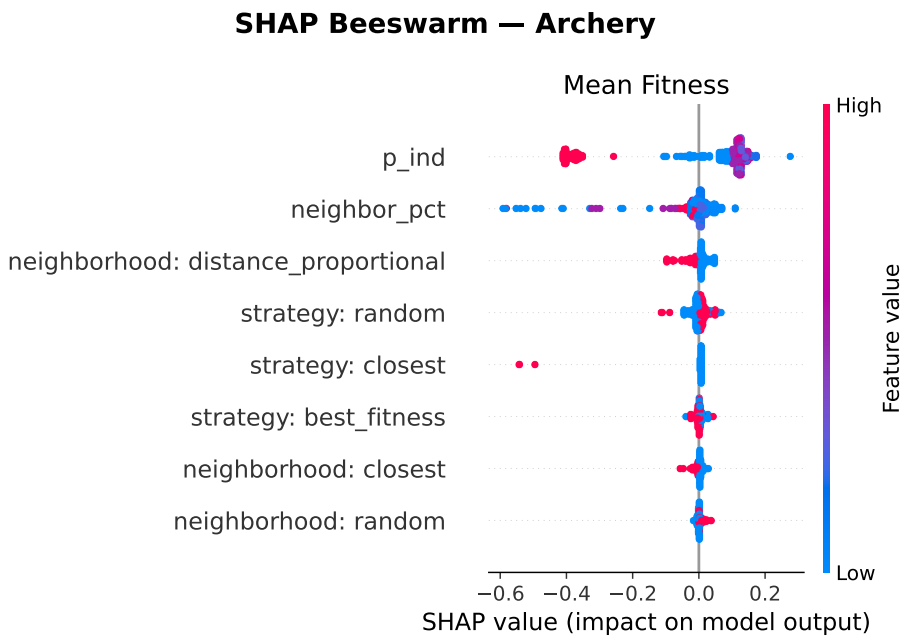

Fig. 1: SHAP beeswarm plot for the archery domain. Each row is a hyperparameter; each dot is a MONET run. Color indicates the feature value (red = high, blue = low); horizontal position indicates the SHAP value.

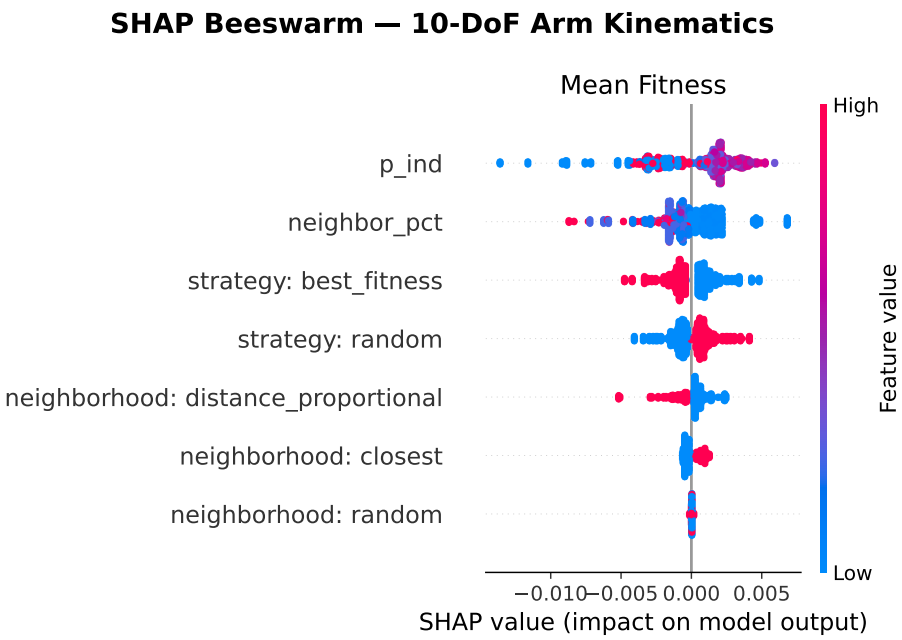

Fig. 2: SHAP beeswarm plot for the arm domain. Panels as in Figure 1.

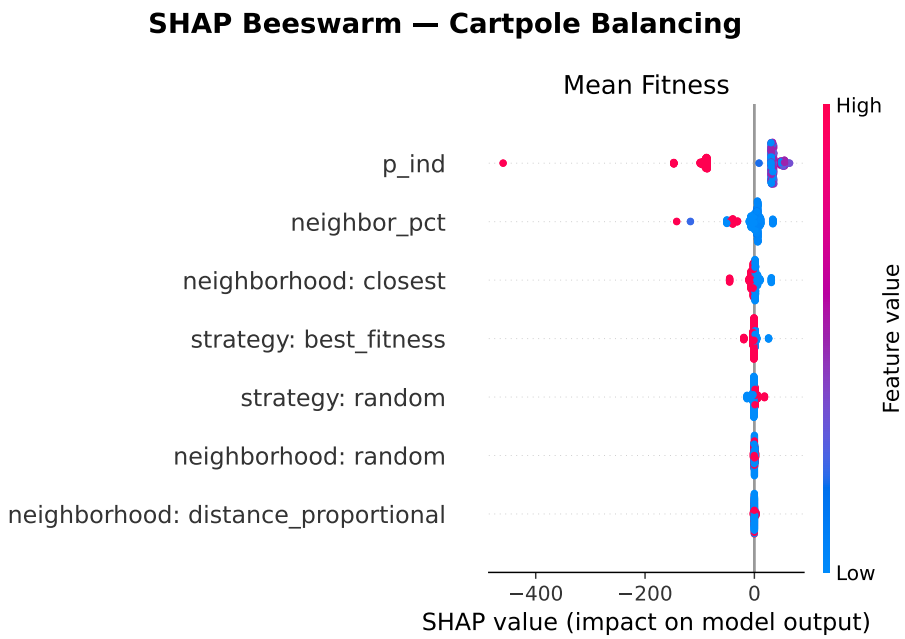

Fig. 3: SHAP beeswarm plot for the cartpole domain. Panels as in Figure 1.

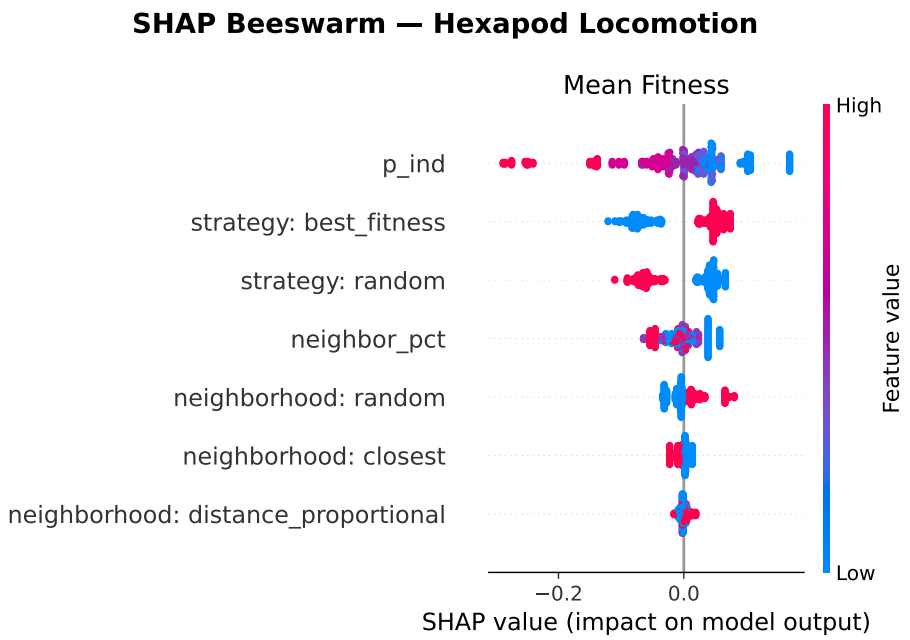

Fig. 4: SHAP beeswarm plot for the hexapod domain. Panels as in Figure 1.

4.2 SPOT-Tuned Configurations

Table 2 lists the per-domain configurations found by the SPOT hyperparameter tuner.

Table 2: SPOT-tuned MONET configurations per domain.

Domain	**strategy**	**neighborhood**	p_{ind}	**neighbor_pct.**
Archery	best_fitness	closest	0.0	2%
Arm	random	closest	0.3	1%
Cartpole	best_fitness	random	0.5	1%
Hexapod	best_fitness	random	0.0	1%

5 MONET Hyperparameter Descriptions

Table 3 summarizes the four hyperparameters varied in the sensitivity analysis. All other algorithmic parameters (mutation operator, crossover operator, initialization strategy) are held fixed throughout. See Algorithm **??** for the points at which each parameter enters the loop.

Table 3: MONET hyperparameters and their roles.

Parameter	Values	Description
p_{ind}	0, 0.3, 0.5, 0.7, 1	Probability that a given evaluation step performs *individual learning* (mutation of the focal task's own elite) rather than *social learning* (crossover with a neighbor's elite). At $p_{\text{ind}} = 0$ only social learning occurs; at $p_{\text{ind}} = 1$ only individual learning occurs.
strategy	random, best_fitness	Neighbor selection strategy during social learning. `random`: uniform random selection among all neighbors. `best_fitness`: deterministically select the neighbor with the highest current fitness.
neighborhood	random, closest, distance_prop.	Method for constructing the neighbor set of each task. `closest`: the N nearest tasks by Euclidean distance in task space. `random`: N tasks sampled uniformly at random (sorted by similarity after sampling). `distance_proportional`: N tasks sampled *without replacement* with probability proportional to task-space similarity $s(\boldsymbol{\tau}_i, \boldsymbol{\tau}_j) = 1/(1 + \lVert\boldsymbol{\tau}_i - \boldsymbol{\tau}_j\rVert)$; this softly biases the neighborhood toward nearby tasks while retaining a chance of including distant ones.
neighbor_pct.	1%, 5%, 10%	Fraction of the total task set used as the neighborhood size $N = \lceil \texttt{neighbor_percentage} \cdot \lvert\mathcal{V}\rvert \rceil$. For instance, with 5 000 tasks and 5%, each task has $N =$ 250 neighbors. Controls graph sparsity: lower values yield sparser graphs with more localized knowledge transfer.

6 Hyperparameter Configuration Ranking

Table 4 lists the ten highest-ranked MONET hyperparameter configurations (across the 2–4 domains on which they were run) by combined z-score, computed as the mean of the within-domain z-scores of final mean fitness and AUC. Table 5 shows the five worst-ranked configurations for comparison. Only configurations evaluated on at least two domains are included in the ranking. The counts $n_{\text{Arm}}, n_{\text{Archery}}, n_{\text{Cartpole}}, n_{\text{Hexapod}}$ report the number of deduplicated seed runs per domain.

Two patterns are clear from Table 4: (i) the `closest` neighborhood dominates the top of the ranking, and (ii) small neighborhood sizes ($\leq 2\%$ of the task set) are preferred. Poorly-performing configurations (Table 5) combine the `distance_proportional` neighborhood with either very high p_{ind} (near pure individual learning) or very large neighborhoods, both of which disrupt local knowledge transfer.

Table 4: Top 10 MONET configurations by combined z-score ($n_{\text{domains}} \geq 2$).

strategy	neighborhood	nbr.%	p_{ind}	n_{Arm}	n_{Arc}	n_{Cart}	n_{Hex}	z_{FF}	z_{AUC}	combined
best_fitness	closest	1.5	0.30	20	20	0	0	+0.58	+1.01	**+0.80**
best_fitness	closest	1.0	0.30	22	22	0	5	+0.60	+0.96	+0.78
best_fitness	closest	2.0	0.30	20	20	0	0	+0.55	+0.99	+0.77
best_fitness	closest	2.0	0.50	20	39	5	0	+0.62	+0.90	+0.76
best_fitness	closest	1.5	0.70	5	20	0	0	+0.62	+0.81	+0.72
best_fitness	closest	2.0	0.70	5	20	0	0	+0.62	+0.82	+0.72
random	closest	2.0	0.00	20	19	4	0	+0.66	+0.75	+0.71
random	closest	1.5	0.00	20	19	5	0	+0.66	+0.75	+0.70
best_fitness	closest	1.0	0.50	40	41	5	5	+0.56	+0.82	+0.69
random	closest	1.0	0.00	40	40	5	3	+0.64	+0.73	+0.68

Table 5: Bottom 5 MONET configurations by combined z-score ($n_{\text{domains}} \geq 2$).

strategy	neighborhood	nbr.%	p_{ind}	n_{Arm}	n_{Arc}	n_{Cart}	n_{Hex}	z_{FF}	z_{AUC}	combined
best_fitness	distance_prop.	5.0	1.00	20	20	0	2	−1.22	−1.86	−1.54
best_fitness	distance_prop.	10.0	0.00	20	20	0	16	−2.51	−0.65	−1.58
random	closest	34.3	0.98	2	1	1	0	−1.19	−1.97	−1.58
best_fitness	distance_prop.	5.0	0.00	20	20	0	3	−3.00	−0.93	−1.96
best_fitness	distance_prop.	41.5	0.99	2	2	2	0	−1.97	−2.23	**−2.10**

7 Code Availability

Table 6 lists the reference papers and public code repositories for MONET and the three baselines.

Table 6: Reference papers and public code repositories for MONET and the baseline algorithms.

Algorithm	**Paper**	**Code**
MONET (ours)	This paper	[7]
MAP-Elites	[3, 9]	[8]
MT-ME	[10]	[8]
PT-ME	[1]	[2]

References

1. Anne, T., Mouret, J.B.: Parametric-Task MAP-Elites. In: Proceedings of the Genetic and Evolutionary Computation Conference. pp. 68–77 (Jul 2024). https://doi.org/10.1145/3638529.3653993

2. Anne, T., Mouret, J.B.: Parametric-Task MAP-Elites. https://github.com/hucebot/Parametric-Task_MAP-Elites (2024)
3. Cully, A., Clune, J., Tarapore, D., Mouret, J.B.: Robots that can adapt like animals. Nature **521**(7553), 503–507 (2015). https://doi.org/10.1038/nature14422, https://doi.org/10.1038/nature14422
4. Deb, K.: Multi-Objective Optimization Using Evolutionary Algorithms. Wiley, Chichester, UK (2001)
5. Deb, K., Agrawal, R.B.: Simulated binary crossover for continuous search space. Complex Systems **9**(2), 115–148 (1995)
6. Feller, W.: An Introduction to Probability Theory and Its Applications, vol. 1. Wiley, 3 edn. (1968)
7. Hatzky, J., Bartz-Beielstein, T., Eiben, A.E., Yaman, A.: Repository for MONET: Multi-Task Optimization over Networks of Tasks. https://github.com/ju2ez/MONET/tree/PPSN-MONET-introduction (2026)
8. Mouret, J.B.: pymap_elites: Python reference implementation of MAP-Elites and Multi-Task MAP-Elites. https://github.com/resibots/pymap_elites
9. Mouret, J., Clune, J.: Illuminating search spaces by mapping elites. CoRR **abs/1504.04909** (2015), http://arxiv.org/abs/1504.04909
10. Mouret, J.B., Maguire, G.: Quality diversity for multi-task optimization. In: Proceedings of the 2020 Genetic and Evolutionary Computation Conference. pp. 121–129. ACM, Cancún Mexico (Jun 2020). https://doi.org/10.1145/3377930.3390203